



NexFlow — Documentation Bundle





Contents

NexFlow — Documentation Bundle

Index

Re-rendering the PDFs

Drift check

System dependencies

Conventions

Repo paths referenced across these docs



The engineering-, design-, and product-facing reference for building, deploying, and operating **NexFlow** — the AI-native B2B CRM and sales operating system for the GCC.

Every document is shipped both as Markdown (for in-repo reading and version control) and as a branded PDF (for distribution to non-technical stakeholders).

Index

#	DOCUMENT	WHAT IT COVERS
01	Brand, Palette & Motion Guide — PDF	Chameleon DNA palette, the diamond-mark logo, Living Mesh background, motion conventions, the two motion-logo HTML files.
02	Full Tech Stack — PDF	Real, current stack of every artifact (web, mobile, API, libs) with version numbers and a stack-reconciliation appendix vs. the feasibility study.
03	Component Inventory — PDF	Every meaningful component in the web and mobile apps, grouped by the 8 TopBar mega-dropdowns.
04	Deployment Plan — PDF	dev / staging / prod, Replit Deployments path, AWS me-south-1 (Bahrain) residency path, DB migrations, secrets, observability, backup & DR, costs, release checklist.
05	In-App Documentation — All Four Roles — PDF	What each role (CEO, Sales Manager, Sales Team, Marketing) sees, modules they live in, step-by-step workflows, AI features, KPIs.
06	Phase 2 — Client Management Spec (Atlas) — PDF	Forward-looking spec for the post-sale relationship intelligence module.
07	Phase 3 — Client Onboarding Spec (Threshold) — PDF	Forward-looking spec for the AI-guided client onboarding & go-live module.

Re-rendering the PDFs

From the repo root:

```
pnpm run docs:pdf
```

That shells out to `python3 docs/render_pdf.py`, which reads every `docs/*.md` (plus `docs/business/*.md` and `docs/investor/*.md`) and renders a branded PDF — NexFlow header, page numbers, palette accents, table of contents — using WeasyPrint. Output goes back into the same folder as the source markdown.



Drift check

To verify every markdown source has an up-to-date PDF sibling (useful as a pre-commit hook or CI check):

```
pnpm run docs:pdf:check
```

It exits non-zero if any `*.md` is newer than its matching `*.pdf` (or the PDF is missing). When that happens, run `pnpm run docs:pdf` and commit the regenerated PDFs alongside your markdown edits.

System dependencies

WeasyPrint needs the following native libraries available on the host: `pango`, `cairo`, `harfbuzz`, `fontconfig`, `freetype`, `gdk-pixbuf`, `glib`. They are already installed in this Replit environment; on a fresh clone, install them via your platform's package manager before running `pnpm run docs:pdf`.

Conventions

- The codebase is the authoritative source for technical decisions. When the original feasibility study disagrees (Next.js vs. Vite, Prisma vs. Drizzle, NestJS vs. Express), `02-tech-stack.md` §13 reconciles.
- Color values, motion durations, and brand assets in any new artifact must follow `01-brand-palette-motion.md`.
- Phase 2 (Atlas) and Phase 3 (Threshold) specs are **forward-looking** — they describe modules that are not yet built.

Repo paths referenced across these docs

<code>artifacts/nexflow/</code>	React + Vite web app
<code>artifacts/mobile/</code>	Expo / React Native mobile app
<code>artifacts/api-server/</code>	Express 5 API
<code>artifacts/marketing-demo-video/</code>	90s animated demo
<code>artifacts/mockup-sandbox/</code>	internal design canvas
<code>lib/db/</code>	Drizzle schema + migrations
<code>lib/api-spec/</code>	OpenAPI + Orval codegen
<code>lib/api-zod/</code>	generated Zod schemas
<code>lib/api-client-react/</code>	generated React Query hooks
<code>attached_assets/</code>	brand source files (logos, motion HTML, feasibility doc)
<code>docs/</code>	this folder
<code>docs/assets/</code>	logo + motion files mirrored for PDF rendering